Numerical Steps -

Mini-Batch GD: Numerical Example :

Mini-Batch GD: Numerical Example

- Problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.1$

Iter	Loss	∇_{w_1}	∇_{w_2}	$w_1 \leftarrow w_1 - \eta\nabla_{w_1}$	$w_2 \leftarrow w_2 - \eta\nabla_{w_2}$
0	32.00	8.0	16.0	$4 - 0.1(8) = 3.20$	$2 - 0.1(16) = 0.40$
1	10.88	6.4	3.2	$3.2 - 0.1(6.4) = 2.56$	$0.4 - 0.1(3.2) = 0.08$
2	6.58	5.1	0.64	$2.56 - 0.1(5.1) = 2.05$	$0.08 - 0.1(0.64) = 0.016$
3	4.21	4.1	0.13	$2.05 - 0.1(4.1) = 1.64$	$0.016 - 0.1(0.13) = 0.003$

Gradient is = $2w_1$ + (4*2=8)w_2

Iter 0: w1=4, w2=2, L=(4^2)+4(2^2)=16+16=32, **∇w1=2(4)=8**, **∇w2=8(2)=16**, w1=4−0.1(8)=3.2, w2=2−0.1(16)=0.4

Iter 1: w1=3.2, w2=0.4, L=(3.2^2)+4(0.4^2)=10.24+0.64=10.88, ∇w1=2(3.2)=6.4, ∇w2=8(0.4)=3.2, w1=3.2−0.1(6.4)=2.56, w2=0.4−0.1(3.2)=0.08

Iter 2: w1=2.56, w2=0.08, L=(2.56^2)+4(0.08^2)=6.5536+0.0256=6.5792, ∇w1=2(2.56)=5.12, ∇w2=8(0.08)=0.64, w1=2.56−0.1(5.12)=2.048, w2=0.08−0.1(0.64)=0.016

Iter 3: w1=2.048, w2=0.016, L=(2.048^2)+4(0.016^2)=4.194304+0.001024=4.195328, ∇w1=2(2.048)=4.096, ∇w2=8(0.016)=0.128, w1=2.048−0.1(4.096)=1.6384, w2=0.016−0.1(0.128)=0.0032

Issue: w2 takes large steps (Δw2 =−1.6) compared to w1 (Δw1 =−0.8)
Learning rate is 0.1 so w1=4**−0.1(8)**, so **Δw1 = −0.8,** same for Δw2 =−1.6

Momentum: Numerical Example

- Same problem, reduced LR: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\beta = 0.9$

Iter	Loss	$[\nabla w_1, \nabla w_2]$	$v_1 \leftarrow \beta v_1 + w_1$	$v_2 \leftarrow \beta v_2 + w_2$	w_1	w_2
0	32.00	$[8, 16]$	8	16	3.6	1.2
1	18.72	$[7.2, 9.6]$	$7.2 + 7.2 = 14.4$	$14.4 + 9.6 = 24$	2.88	0
2	8.29	$[5.76, 0]$	$13.0 + 5.76 = 18.7$	$21.6 + 0 = 21.6$	1.94	-1.08
3	8.43	$[3.88, -8.64]$	$16.9 + 3.88 = 20.7$	$19.4 - 8.64 = 10.8$	0.91	-1.62

- Still oscillates! $w_2 : 2 \rightarrow 1.2 \rightarrow 0 \rightarrow -1.08 \rightarrow -1.62$
- Key lesson: Momentum amplifies steps - needs smaller η than vanilla GD
- For remaining examples: We'll use $\eta = 0.05$ for fair comparison

Initial velocity is 0 for both w1 and w2

Iter 0: w1=4, w2=2, L=(4^2)+4(2^2)=16+16=32, ∇=[2*(4)=8,8*(2)=16], v=[0.9(0)+8=8, 0.9(0)+16=16], w1=4−0.05(8)=3.6, w2=2−0.05(16)=1.2

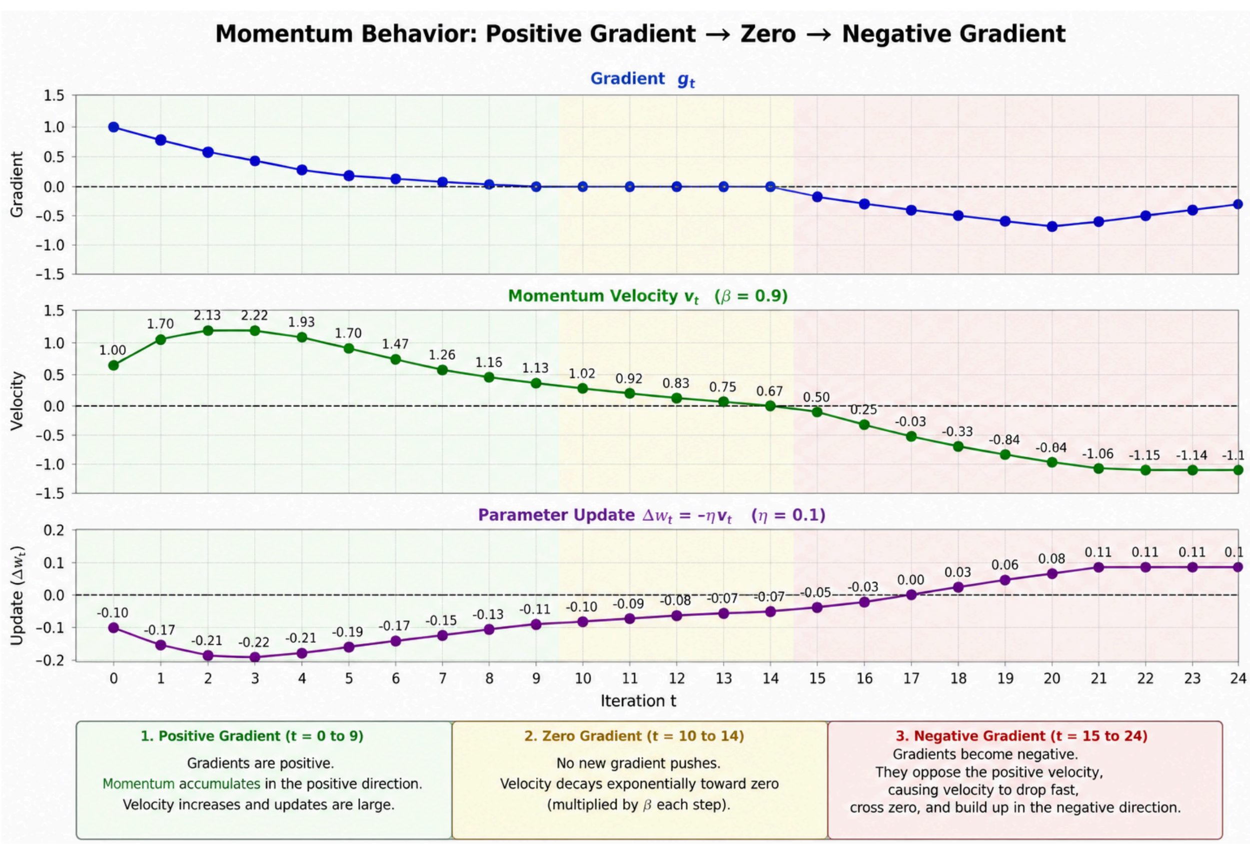
Iter 1: w1=3.6, w2=1.2, L=(3.6^2)+4(1.2^2)=12.96+5.76=18.72, ∇=[2(3.6)=**7.2**, 8(1.2)=**9.6**], v=[0.9(8)+**7.2**=14.4, 0.9(16)+**9.6**=24], w1=3.6−0.05(14.4)=2.88, w2=1.2−0.05(24)=0

Iter 2: w1=2.88, w2=0, L=(2.88^2)+0=8.2944, ∇=[2(2.88)=5.76, 0], v=[0.9(14.4)+5.76=18.72, 0.9(24)+0=21.6], w1=2.88−0.05(18.72)=1.944, w2=0−0.05(21.6)=−1.08

Iter 3: w1=1.944, w2=−1.08, L=(1.944^2)+4(−1.08^2)=3.779+4.6656=8.4446, ∇=[2(1.944)=3.888, 8(−1.08)=−8.64], v=[0.9(18.72)+3.888=20.736, 0.9(21.6)−8.64=10.8], w1=1.944−0.05(20.736)=0.9072, w2=−1.08−0.05(10.8)=−1.62

Key lesson: Momentum amplifies steps -

Ravine Problem - A ravine is: steep in one direction and flat in another just like w2 and w1.



Nesterov Accelerated Gradient (NAG) - Example

Given: $L(w1,w2)=w_1^2+4w_2^2$, $\nabla=[2w1, 8w2]$, $\eta=0.05$, $\beta=0.9$, $v0=[0,0]$, starting point = [4,2]

In Nesterov Accelerated Gradient, we do not compute the gradient at the current point w. Instead, we compute it at a look-ahead point:

$\hat{W} = w-\beta(v)$
So: w = current position
v = velocity (momentum)
$\beta(v)$ = how far momentum is pushing you forward
$\hat{W}$ = predicted next position

Iter 0: $w=(4,2)$, $v=(0,0)$, $\hat{w}=(4,2)$, $L=4^2 + 4(2^2)=16+16=32$, $\nabla(\hat{w})=(2(4),8(2))=(8,16)$, $v=(0.9(0)+8,0.9(0)+16)=(8,16)$, $w=(4,2)-0.05(8,16)=(3.6,1.2)$

Iter 1: $w=(3.6,1.2)$, $v=(8,16)$, $\hat{w}=(3.6-0.9(8),1.2-0.9(16))=(-3.6,-13.2)$, $L=(-3.6)^2+4(-13.2)^2=12.96+696.96=709.92$, $\nabla(\hat{w})=(2(-3.6),8(-13.2))=(-7.2,-105.6)$, $v=(0.9(8)+(-7.2),0.9(16)+(-105.6))=(0,-91.2)$, $w=(3.6,1.2)-0.05(0,-91.2)=(3.6,5.76)$

Iter 2: $w=(3.6,5.76)$, $v=(0,-91.2)$, $\hat{w}=(3.6-0.9(0),5.76-0.9(-91.2))=(3.6,87.84)$, $L=3.6^2+4(87.84^2)=12.96+30864.51\approx30877.47$, $\nabla(\hat{w})=(2(3.6),8(87.84))=(7.2,702.72)$, $v=(0.9(0)+7.2,0.9(-91.2)+702.72)=(7.2,620.64)$, $w=(3.6,5.76)-0.05(7.2,620.64)=(3.24,-25.272)$

Iter 3: $w=(3.24,-25.272)$, $v=(7.2,620.64)$, $\hat{w}=(3.24-0.9(7.2),-25.272-0.9(620.64))=(-3.24,-583.848)$,
$L=(-3.24)^2+4(-583.848)^2=10.50+1363806.8\approx1363817.3$,
$\nabla(\hat{w})=(2(-3.24),8(-583.848))=(-6.48,-4670.784)$,
$v=(0.9(7.2)+(-6.48),0.9(620.64)+(-4670.784))=(0,-4103.208)$,
$w=(3.24,-25.272)-0.05(0,-4103.208)=(3.24,180.888)$

Iter 4: $w=(3.24,180.888)$, $v=(0,-4103.208)$,
$\hat{w}=(3.24-0.9(0),180.888-0.9(-4103.208))=(3.24,3872.776)$,
$L=(3.24)^2+4(3872.776^2)=10.50+59967862.5\approx59967873.0$,
$\nabla(\hat{w})=(2(3.24),8(3872.776))=(6.48,30982.208)$,
$v=(0.9(0)+6.48,0.9(-4103.208)+30982.208)=(6.48,27291.328)$,
$w=(3.24,180.888)-0.05(6.48,27291.328)=(2.916,-1233.678)$

Important observation (very important conceptually)

From Iter 2 onward:

- w2 explodes
- L explodes
- NAG becomes unstable

Reason: $\eta(0.05)$ is too large for curvature 8 in w2

So the method is mathematically correct, but numerically unstable.

Solution - Step 1: Understand stability condition, For quadratic functions, stability depends on the largest curvature: w1 curvature = 2, w2 curvature = 8 ← dominant
So the safe step size must satisfy roughly: $\eta < 1 / 8 = 0.125$
But with momentum ($\beta = 0.9$), effective dynamics amplify updates, so we use a smaller safe value.

Step 2: Choose stable η. A good stable choice is: $\eta = 0.02$ (safe for NAG with $\beta = 0.9$)
Step 3: Why this works - Reduces overshooting in w2 direction. Keeps velocity from exploding. Ensures w2 converges instead of diverging. Still allows smooth acceleration from momentum

Final answer: Use: $\eta = 0.02$, $\beta = 0.9$
This makes NAG: stable, convergent, smoothly decreasing L(w1,w2)

Gradient Clipping

Take gradient g = (6, 8), so its norm is $\|g\| = \sqrt{(6^2 + 8^2)} = \sqrt{(36 + 64)} = \sqrt{100} = 10$. Let $\tau = 5$.
Norm-based clipping: Since $\|g\| > \tau$, we scale the whole vector.
Scale factor = $\tau / \|g\| = 5 / 10 = 0.5$
So clipped gradient = 0.5 (6, 8) = (3, 4)
Here direction is preserved and only magnitude is reduced.
Value-based clipping: We clip each component independently to the range [−5, 5].
So 6 becomes 5 and 8 becomes 5.
Clipped gradient = (5, 5)
Here the direction changes because components are altered unevenly.
Summary: Norm clipping gives (3, 4) preserving direction, value clipping gives (5, 5) which changes direction.